



Jose Parreño Garcia <joparga3@gmail.com>

[TEST g7b - Paid] Claude Code agents: what they actually are

1 mensaje

Jose Parreño Garcia from Senior Data Science Lead <joseparreogarcia@substack.com>
Para: joparga3@gmail.com

4 de abril de 2026 a las 15:17

[View in browser](#)

The Senior Data Science Lead newsletter

Hi! It's Jose from Senior Data Science Lead newsletter, where I write about senior data science leadership, sharing hard-earned lessons from building teams, systems, and real-world machine learning.

Because you are a paid subscriber, you are directly supporting this work — and you get full access to the practical tools built alongside the writing:

- ***The full Data Storytelling course, focused on turning analysis into insights that land with executives. Because today, anyone can build a chart with AI, but very few people can build the right chart.***



Master Data Storytelling

JOSE PARREÑO GARCIA · NOVEMBER 27, 2025

[Read full story](#)

- ***50+ Notion templates for hiring, onboarding, mentoring, and planning built for Data and Tech Leads. As a paid subscriber, you get 50% off with code PAIDSUBS-03PL.***



50+ Data Science Templates

JOSE PARREÑO GARCIA · APRIL 8, 2025

[Read full story](#)

And as always, thank you for supporting the newsletter — your support is what keeps it independent, practical, and focused on real-world impact.

Claude Code agents: what they actually are

A ground-up guide to subagents In Claude Code for practitioners who want to understand the feature before building with it.

JOSE PARREÑO GARCIA

APR 4 · PAID · DRAFT





READ IN APP ↗

The first time I saw Claude spawn a subagent in my newsletter pipeline, I assumed it was just running a better-scoped prompt. I watched it work through a dozen files, and return a clean summary. It was actually pretty cool to watch, but I didn't fully understand what had happened. And, that really bothered me.

Most people encounter Claude Code agents and think its a skill with extra steps (I have even heard its a "smarter skill", because it is an "agent"). Something that behaves more autonomously because the instructions are written differently.

That mental model is wrong. **Agents** are indeed a prompt similar to a skill, but, the key difference is that they do not run inside your main conversation. They run in their own context window, with their own system prompt, their own tool access, and their own permissions. They work separately and return a result.

This post explains agents from the ground up — what they are, how to build one, and when they're worth the investment. It assumes you're already using Claude Code and want to understand this feature properly before touching it.

✓ Subscribed

What this post covers

- **The naming problem: what do people mean by “Claude Code agents”?** Claude Code uses several “agent” terms that mean different things. Getting this straight first prevents confusion throughout.
- **Why Claude Code subagents exist.** The core reason is context management.
- **Building your first custom subagent in Claude Code.** The minimal file, every field that matters, and a concrete walkthrough.
- **Agents vs skills: the confusion everyone runs into.** The most important boundary in the Claude Code primitive stack — and what the two look like on the same task.
- **Best practices for Claude Code subagents.** What the description field is actually doing, why tool restriction matters beyond security, and what the practitioner community has converged on.
- **A quick look at agent teams.** A quick look at the multi-session layer and when it’s worth the tradeoff.
- **When agents are the wrong tool.** The boundaries that keep things maintainable.

Let’s get started!

The naming problem: what do people mean by “Claude Code agents”?

Before anything else: the word “agents” in Claude Code means at least 4 different things, and conflating them makes the feature much harder to reason about. I might be a bit pedantic with this section, but I felt it helped me really map vocabulary to assets that I can use.

1. **Claude Code itself is an agentic environment.** Anthropic describes Claude Code as an agentic coding assistant — one that has tools, an execution loop, and the ability to plan and act across multiple steps without waiting to be prompted at each one. That’s the baseline. It’s already agentic before you configure anything.
2. **Subagents are the custom in-product agent primitive.** When a practitioner says “I’m building an agent in Claude Code,” they almost always mean a **subagent**. The [official documentation](#) defines them as “specialized AI assistants that handle specific types of tasks. Each subagent runs in its own context window with a custom system prompt, specific tool access, and independent permissions.”
3. **Agent teams are a separate, multi-session feature.** The [agent teams documentation](#) draws this line clearly: “Subagents work within a single session; agent teams coordinate across separate sessions.” Agent teams are independent Claude Code instances that communicate via a shared task list and direct peer messaging. They are experimental, disabled by default, and require an environment flag to enable. They are not “more subagents” — they are a different layer of the architecture.
4. **The Claude Agent SDK is a fourth layer** — the same tools and agent loop that power Claude Code, exposed as a programmable Python and

TypeScript library. It exists outside the CLI, for teams building custom agent applications. Not the subject here.

With this definition, this post is about subagents (ie, point 2 above). I am eager to write a set of posts on agent teams, but I definitely want to cover the basics of 1 subagent in detail.

Why Claude Code subagents exist.

The main reason why subagents exist is because LLM models have a finite context. **Sooner or later, you *will* hit the context limit.**

This means that, unless you do something about it, you will suffer context limits again, and again, and again. Anthropic's [post on context engineering](#) frames the problem precisely: “*Context engineering is the art and science of curating what will go into the **limited** context window.*” Every tool call, every file read, every partial analysis — it all lands in context and stays there. As a session grows, the window fills with intermediate work that was necessary to produce a result but is no longer useful to reference. That noise degrades performance (this is called [context rot](#) - I'll probably write a blog post on techniques to minimise this). Claude has less room to work, and more irrelevant content competing for its attention.

Subagents exist to help solve this problem.

The [Extend Claude Code documentation](#) states it directly: “*The subagent might read dozens of files or run extensive searches, but your main conversation only receives a summary. Since subagent work doesn't*

consume your main context, this is also useful when you don't need the intermediate work to remain visible."

Here is what that looks like in practice. Imagine you ask Claude to investigate why a test suite is failing. Doing that investigation in the main session means: read 20 files, grep for error patterns, check test configuration, scan recent commits. All of that lands in your context. By the time the investigation is done, you've consumed several thousand tokens of intermediate work, and most of it has no bearing on what you do next.

A subagent changes the shape of that problem. It runs the investigation in its own context window, consuming its own token budget. You get back: "The test failures are caused by a missing environment variable in the CI config. Here are the three affected tests and the fix." Clean. The main thread never absorbed the search context bloat.

Context isolation is the primary reason. But there are 3 secondary reasons worth naming:

1. **Specialisation.** A subagent has its own system prompt. A focused code reviewer with instructions specific to your team's standards is more reliable than a general assistant trying to hold those standards alongside everything else it knows.
2. **Constraint enforcement.** You can specify exactly which tools a subagent can use. A read-only research agent that cannot write files or run shell commands is a materially different risk profile from the main assistant. The constraints are declared and enforced.

3. **Cost control.** Subagents can run on a different model. Routing lightweight tasks to Haiku while the main session runs on Sonnet is a practical cost handle most practitioners don't discover until they've been surprised by their bill.

Building your first custom subagent in Claude Code.

A subagent is a Markdown file with YAML frontmatter.

It lives in 1 of 2 places:

- `.claude/agents/` in your project directory — for agents specific to a codebase. Check these into version control so your team can share and improve them.
- `~/.claude/agents/` in your home directory — for personal agents available across all your projects.

Let's speak about the YAML frontmatter

One thing that people don't really pay a lot of attention to is the YAML frontmatter for subagents. This frontmatter is the metadata that Claude Code will in memory in your main session to understand that the agent is about. In other words, it serves as a summary of your agent.

There are 2 required fields and a few extra optional ones. Let's see these in a simple example.


```
---  
name: code-reviewer  
description: Reviews code changes for quality, security, and best  
practices.  
            Use proactively after any significant code change.  
tools: Read, Glob, Grep  
model: sonnet  
---
```

You are a code reviewer. When invoked, read the changed files and provide specific, actionable feedback on code quality, security, and best practices. Focus on what matters. Don't restate what the code does – say what's wrong or good about it. Flag security issues first, then logic problems, then style.

Let's walk through what each part is doing.

- name is the unique identifier. Lowercase, hyphens only. It's how the agent appears in logs and how Claude refers to it internally.
- description is **the most important field**. Claude reads this to decide whether to delegate a task here. Write it behaviourally: what the agent does, and when to use it. The phrase “use proactively” is a documented signal [1] — it tells Claude to activate this agent without waiting to be

explicitly asked. Leave that phrase out if you want the agent to respond only on direct request.

- `tools` restricts what this agent can access. This reviewer can read files and search — it cannot edit, run shell commands, or make network calls. That restriction is intentional. A read-only reviewer cannot accidentally modify what it's reviewing. The [official documentation](#) explicitly recommends granting only the tools an agent actually needs, both for security and to make its behaviour predictable.
- `model` routes this agent to a specific model. Options: `sonnet`, `opus`, `haiku`, or a full model ID like `claude-opus-4-6`. Omit it and the agent defaults to `inherit` (basically, inheriting the same model as the main session).
- The body of the file — everything below the frontmatter — becomes the agent's system prompt. It receives this, the working directory, and nothing else from the main session.

The [full set of optional frontmatter fields](#) covers more parameters that will make your agents really really powerful:

`permissionMode`, `maxTurns`, `skills`, `mcpServers`, `effort`, `memory`, `hooks`, `disallowedTools`, `color...`

<code>tools</code>	No	Tools the subagent can use. Inherits all tools if omitted
<code>disallowedTools</code>	No	Tools to deny, removed from inherited or specified list
<code>model</code>	No	Model to use: <code>sonnet</code> , <code>opus</code> , <code>haiku</code> , a full model ID (for example, <code>claude-opus-4-6</code>), or <code>inherit</code> . Defaults to <code>inherit</code>
<code>permissionMode</code>	No	Permission mode: <code>default</code> , <code>acceptEdits</code> , <code>auto</code> , <code>dontAsk</code> , <code>bypassPermissions</code> , or <code>plan</code>
<code>maxTurns</code>	No	Maximum number of agentic turns before the subagent stops
<code>skills</code>	No	Skills to load into the subagent's context at startup. The full skill content is injected, not just made available for invocation. Subagents don't inherit skills from the parent conversation
<code>mcpServers</code>	No	MCP servers available to this subagent. Each entry is either a server name referencing an already-configured server (e.g., <code>"slack"</code>) or an inline definition with the server name as key and a full MCP server config as value
<code>hooks</code>	No	Lifecycle hooks scoped to this subagent
<code>memory</code>	No	Persistent memory scope: <code>user</code> , <code>project</code> , or <code>local</code> . Enables cross-session learning
<code>background</code>	No	Set to <code>true</code> to always run this subagent as a background task . Default: <code>false</code>
<code>effort</code>	No	Effort level when this subagent is active. Overrides the session effort level. Default: inherits from session. Options: <code>low</code> , <code>medium</code> , <code>high</code> , <code>max</code> (Opus 4.6 only)
<code>isolation</code>	No	Set to <code>worktree</code> to run the subagent in a temporary git worktree , giving it an isolated copy of the repository. The worktree is automatically cleaned up if the subagent makes no changes

Screenshot [from Anthropic's documentation](#).

Having understood a simple agent, you might think that it's pretty similar to a Claude Code skill... you are both correct and missing the point. So, let's deep dive into the differences between both.

Where it lives: folder structure

Save the example above as `.claude/agents/code-reviewer.md` in your project. Here's what that looks like:

```
your-project/
├── .claude/
│   ├── agents/
│   │   └── code-reviewer.md      ← your new agent file
│   ├── skills/
│   └── rules/
├── src/
├── tests/
├── README.md
└── package.json
```

Once saved, Claude will detect the agent. Use it by asking Claude to review code changes — it will delegate to `code-reviewer` automatically based on the description match.

For personal agents (available across all projects), move the file to `~/.claude/agents/code-reviewer.md` instead. The format and content stay the same; only the location changes.

Agents vs skills: the confusion everyone runs into

First of all, why are people confused about subagents and skills?

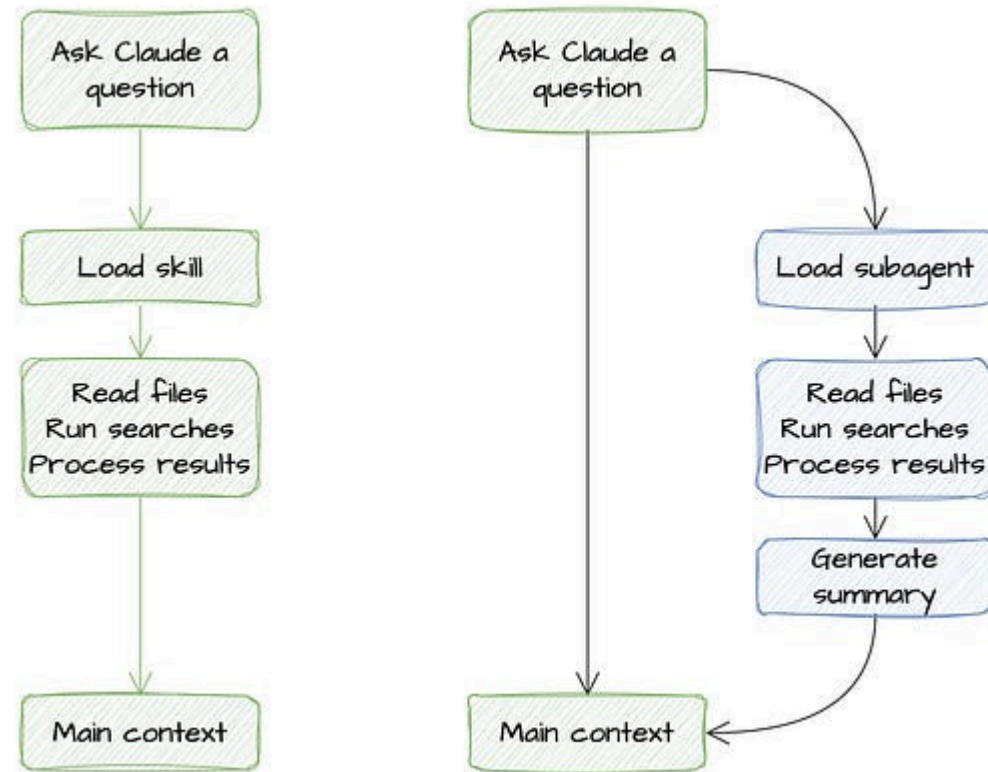
1. They are both setup with a simple folder structure. Inside `.claude/skills` or `.claude/agents`.
2. They both are structured using the frontmatter, with description being the most important parameter to configure.
3. They are both sets of instructions (ie, a prompt). A skill and an agent prompt could be doing exactly the same thing based on a prompt.

Reading these 3 reasons, it does look like structurally, they are really similar.

Where the work happens

However, there is 1 main palpable and visual difference: **where** the work happens, or in the other words, where the tokens are consumed. This is the simplest difference to understand.

- **A skill loads into your current conversation**, its instructions, templates, and reference files become part of the active context. Claude follows them in the same thread, and every step accumulates in your main window.
- **A subagent runs separately in their own context window**. It does its work, then your main conversation gets back a result.



The table below shows the differences more clearly.

Differences of skills vs subagents

	Skills	Subagents
What it provides	Reusable knowledge or workflow logic	Task delegation to an isolated worker
Runs in	Main conversation context	Own context window
Main context cost	Skill content is loaded	Only the returned summary
Invoked by	Slash command or automatic selection	Claude delegating based on description
Best for	Repeatable workflows, domain knowledge, reference material	Context-heavy investigation, specialist execution, constrained work

Created with Datawrapper

Another interesting difference: tuning parameters

Earlier in the post, we saw a screenshot of all the parameters that an agent can have in their frontmatter. It was a long list. On the contrary, **skills have a much more limited list** (and some different)

This should already give us a hint of how much more control and much more tuned can agents be compared to skills. Don't get me wrong, I am not advocating for everything to be an agent. But maybe what we need to rethink is the mental model to design robust systems.

A concrete example makes this easier to see.

Say you want to enforce your team's code review standards.

If you want Claude to know those standards throughout a session — apply them when writing code, reference them when making suggestions, flag violations inline — that's a skill. Write the standards once, and Claude loads them into context when relevant. They are available across everything you do in that session.

Now, if you want to review 40 files in one pass without your main context filling up with file reads, intermediate comments, and search results, that's a subagent. You configure a code-reviewer agent. It runs the review in its own context window and returns a structured summary (or even, you could spawn 40 subagents in parallel for each of the files). Your main conversation receives just the result, not the work that produced it.

A sentence that captures it cleanly:

A skill changes what Claude knows or how it should approach a workflow.

A subagent changes who is doing the work, in what context, and with what tools.

Best practices for Claude Code subagents

The description is the routing interface

This deserves more emphasis than it usually gets.

Claude reads subagent descriptions to decide when to delegate. It **does not read** the body before making that decision. So, if you write specifics in the body of the prompt, Claude Code will never see this until the subagent is spawned. This means that **you need to give a really good description** to Claude so that it knows when the subagent is needed or worth spawning.

- Write in specific, behavioural terms.
- Describe what the agent does and when to use it.
- Include the kinds of tasks it handles well and, if useful, the kinds it shouldn't handle.
- The phrase “use proactively after code changes” is a tested pattern — the documentation mentions it specifically as a way to encourage delegation without waiting for an explicit request. Leave it out for agents you want activated only on direct invocation.
- One thing to avoid: writing descriptions that describe the agent's identity (“You are an expert code reviewer with 20 years of experience...”) rather than its activation conditions. The former belongs in the system prompt. The description is for the routing model, not for the agent itself.

A concrete contrast.

- **Too vague description:** “Helps with code quality and review tasks.”
- **Routing description:** “Reviews staged code changes for security issues, logic bugs, and style violations. Use proactively after each significant edit. Does not generate code or make file edits.”

Restrict tools deliberately

An agent that inherits all tools from the main session is an agent you cannot reason about when something goes wrong. Grant only what the task requires.

- A code reviewer needs read access — not write access, not shell execution, not network calls.
- A research agent needs search and fetch — not file editing.
- A test-runner needs Bash execution — not web access.

Tool restriction is both a security practice and a clarity practice. A narrow tool set makes the agent's behaviour predictable. When the agent fails or produces unexpected output, a short list of possible tools makes diagnosis faster. More importantly, it makes the agent's failure modes finite and knowable.

The `disallowedTools` field lets you inherit the main session's tool set and then remove specific tools — useful when you want “everything except Bash” or “everything except Write.”

One task per agent

The [sub-agents documentation](#) is direct: each subagent should excel at one specific task. An agent with a description like “reviews code, writes tests, updates documentation, and handles deployment checks” has four descriptions competing in one field. Claude might route to it inconsistently, and the system prompt will try to handle four modes of operation simultaneously.

Narrow the scope. Three separate files, each with a single clear description.

- In a data science context: a model validation agent, a data quality checker, a notebook reviewer.
- In a software context: a code reviewer, a test generator, a documentation updater.

The principle is the same regardless of domain: one job, one file, one description that routes cleanly.

Use the model field as a cost lever

This is the most commonly ignored field among practitioners who haven't been surprised by their usage costs yet.

- Setting `model: haiku` on those agents reduces cost substantially when multiple agents are running in a session.
- Setting `model: opus` on an agent that needs sustained reasoning over a long context window justifies the cost.
- The default (`inherit`) means every agent runs on whatever model the main session uses — which is often the most powerful model, applied equally to tasks that don't need it.

The community signal

The [VoltAgent collection of subagents](#) has catalogued over 100 specialist subagents — language experts, security reviewers, infrastructure agents, debuggers, orchestrators. The [Everything Claude Code repo](#) also has a great collection of agents.

I personally haven't read all the 100+ agents in these codebases, but I have skimmed through their names, and I am sure I wouldn't need even 20% of them. So, the amount of agents is not the notable thing to focus on: it's the structure.

Every agent in the collection is narrow. A TypeScript specialist. A SQL expert. A security auditor. A Kubernetes operator. Nothing tries to handle more than one domain.

That's the practitioner consensus after a year-plus of real usage: many focused agents outperform one general-purpose one.

A quick look at agent teams

Agent teams are worth knowing about, but they're a different feature from subagents.

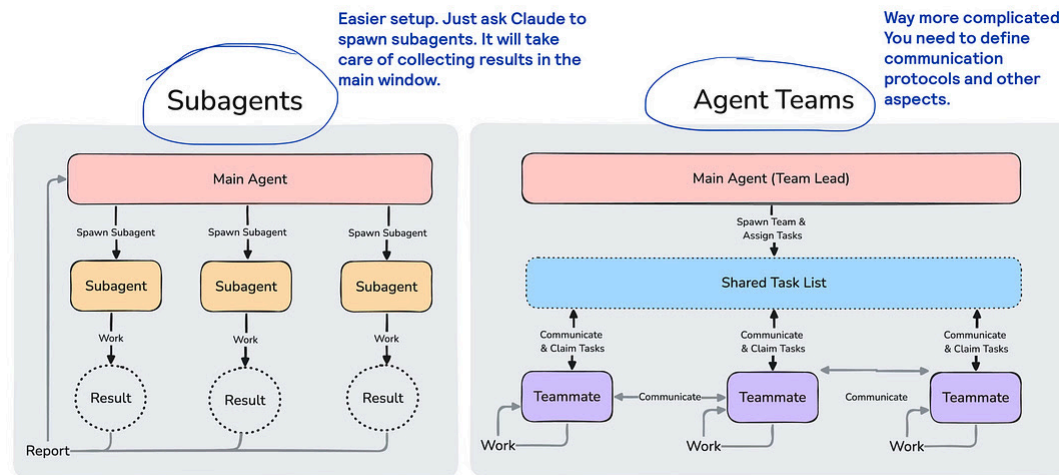
Subagents work within a single session. Your main Claude Code instance spawns them, they do work in isolation, they return results. **You orchestrate.**

Agent teams are independent Claude Code sessions that communicate with each other through a shared task list and direct peer messaging. Each teammate has its own context window, its own session state, and can both assign tasks to and receive tasks from other teammates.

The [agent teams documentation](#) identifies when this pays off: research and review where multiple teammates investigate different angles simultaneously; debugging with competing hypotheses where teammates challenge each

other's theories; new modules or features where each teammate owns a separate component without stepping on the others.

I will not cover agent teams in this post, as their setup is not straightforward. This means that you should not jump on the agents team wagon for FOMO. Agent teams add coordination overhead and consume significantly more tokens — each teammate has its own context window, billed independently. They are experimental and disabled by default, requiring an environment variable to enable.



Screenshot from Anthropic's documentation.

The practical signal is simple: as long as your subagents are reporting back to you and don't need to talk to each other, you don't need agent teams. The moment one subagent needs to hand off findings to another, or challenge another's conclusions, that's when the upgrade earns its cost. Until then, subagents are enough.

When agents are the wrong tool

Anthropic's own guidance from the [building effective agents post](#) is worth taking seriously: *"We recommend finding the simplest solution possible, and only increasing complexity when needed. This might mean not building agentic systems at all."*

That applies directly here. A few scenarios where a subagent adds friction without return:

- **One-off tasks.** Subagents carry authoring overhead: naming, description design, tool selection, testing, maintenance. A task you'll do once doesn't warrant any of that. Write a good prompt. The investment in a subagent only returns when the workflow recurs frequently enough to justify maintaining a definition for it.
- **When context isn't the problem.** The primary reason to use a subagent is to protect your main context from heavy work. If the task is lightweight, your context window isn't filling up, and you don't need specialised tool restrictions — a subagent is adding indirection with no benefit. A well-written prompt in the main session is faster, simpler, and easier to debug.
- **The monolith trap.** One agent trying to cover research, implementation, review, and documentation is not comprehensive — it's untestable and inconsistently selected. The shorter and more specific the description, the more reliably it routes.

The general principle: reach for the simplest tool that solves the actual problem. Add agents when the problem is specifically about context isolation, specialised constraints, or dedicated execution; not because agents seem more powerful.

Closing thoughts

Subagents are a context management tool and an execution boundary. However, most people approach them as a capability upgrade.

The better frame is: a way to make Claude do the right work in the right place. Skills define what Claude knows. Subagents define who does the work, where, and with what constraints.

One agent, configured carefully, is a reasonable afternoon's work. You don't need an orchestration system to get value from the feature. A focused description, a restricted tool set, a single clear job. The complexity can grow from there, but it doesn't have to start there.

Now, I want to hear from you

- If you haven't tried subagents yet: what's the task in your current workflow that you'd most want to delegate to an isolated worker? What's stopping you?
- If you have: what was the first task where the context isolation actually mattered — where running it in the main session would have caused a

real problem?

- How are you handling tool access in practice — per-agent restriction, or inheriting from the main session and trimming from there?

Further reading

If you are interested in more content, here is an article capturing all my written blogs!



All my articles, one convenient place

JOSE PARREÑO GARCIA · OCTOBER 15, 2024

[Read full story](#)

References

[1] [Create custom subagents — Claude Code Docs](#) — Official definition of subagents, supported frontmatter fields, directory locations, delegation behaviour, and best practices including tool restriction and description design.

[2] [Extend Claude Code — Claude Code Docs](#) — The clearest official comparison across Claude Code primitives; explains context isolation as the primary reason for subagents; includes skills vs subagents comparison and combination patterns.

[3] [Orchestrate teams of Claude Code sessions — Claude Code Docs](#) — Defines agent teams, explains how they differ from subagents, lists use cases, and documents experimental status and token cost tradeoffs.

[4] [Effective context engineering for AI agents — Anthropic Engineering](#) — Frames context as a finite, degradable resource and introduces context engineering as the discipline of managing what enters the limited context window.

[5] [Building effective agents — Anthropic Engineering](#) — Anthropic's guidance on when agent systems are warranted; recommends starting simple and distinguishes workflows from agents.

[6] [awesome-claude-code-subagents — VoltAgent / GitHub](#) — A community collection of 100+ specialist subagents demonstrating practitioner convergence on narrow, focused agents organised by domain.

Thanks again for supporting the Senior Data Science Lead newsletter.

Your support makes it possible for me to keep writing openly, while also investing time in the more structured, hands-on work — courses, templates, and tools — that help people apply these ideas in practice. You can find them [here](#).



50+ Data Science Templates

JOSE PARREÑO GARCIA · APRIL 8, 2025

[Read full story](#)



Master Data Storytelling

JOSE PARREÑO GARCIA · NOVEMBER 27, 2025

[Read full story](#)

This newsletter exists in its current form because of readers like you. I am genuinely grateful for that.

Thank you.

 LIKE

 COMMENT

 RESTACK

© 2026 Jose Parreño Garcia
548 Market Street PMB 72296, San Francisco, CA 94104
[Unsubscribe](#)

Get the app

 Start writing